

Trabajo Práctico Integrador

1. Introducción

Para este trabajo práctico, deben desarrollar un backend básico (y reducido) que permita a los usuarios operar en el mercado de acciones bursátiles, bajo las consideraciones que se detallan posteriormente en este documento.

Para los efectos de este Trabajo Práctico, el backend mantiene el registro de los **usuarios** del sistema (quienes pueden comprar y vender **acciones**) y cada uno de estos usuarios tiene un **portfolio** con sus tenencias (o sea, las acciones que posee) y un **saldo** (dinero) en sus cuentas. El saldo se incrementa cuando un usuario decide realizar un ingreso de dinero a la cuenta (y asumimos que los usuarios tienen dinero infinito, pueden ingresar dinero cuando quieran). También asumimos que el saldo en cuenta está siempre en Pesos Argentinos (ARS).

Los usuarios pueden consultar (libremente) la cotización actual de cualquier acción, y las acciones se identifican por su símbolo (por ejemplo: NVDA -> Nvidia Corporation). A esta cotización la deben obtener haciendo uso de un API externa al backend. Pueden usar el API que quieran: Yahoo Finance o cualquier otra.

Cuando un usuario quiere comprar acciones, debe crear una **Orden de Compra** detallando, mínimamente, qué acción quiere comprar, en qué cantidad y cuál es el precio unitario máximo dispuesto a pagar. Ej: El usuario quiere comprar 10 acciones de NVDA a un precio máximo de AR\$ 35.466 por acción. Pero, ¿a quién se le compran las acciones?. Las acciones se le compran a alguien que esté dispuesto a vender las suyas y esta intención de venta se realiza mediante la colocación de una **Orden de Venta** que indica, mínimamente, qué acciones quiere vender, en qué cantidad y cuál es el precio mínimo que acepta por ellas.

Sabiendo que los usuarios pueden colocar órdenes de compra y de venta, una parte crítica del backend es el motor de "emparejamiento" de estas órdenes. El backend, ante una orden de compra, debe buscar si hay órdenes de venta que permitan realizar la compra al usuario. Para el trabajo que deben hacer, vamos a considerar que la orden de compra se acepta o rechaza inmediatamente (no es así en la realidad, y si alguien quiere hacerlo en "background" tiene la libertad para hacerlo). Esto significa que cuando el usuario crea la orden de compra espera respuesta inmediata del endpoint correspondiente con el resultado de la operación.

Tengan especial atención a la posible necesidad de realizar conversiones de monedas. El saldo en dinero de la cuenta está en Pesos Argentinos, algunas acciones pueden tener su cotización en otra moneda y, llegado el caso, se debe hacer la conversión apropiada para que los saldos que se descuenten o acrediten estén en Pesos Argentinos. Aquí pueden utilizar la solución que consideren más conveniente (un API externa, datos "mockeados", una tabla paramétrica, etc.).

2 Especificaciones del Motor de Emparejamiento de Órdenes

1. Como ya se mencionó, el resultado se asume inmediato (la orden de compra se aceptó y se actualizaron todos los saldos y portfolios, o se rechazó). Pero, nuevamente, si un grupo quiere hacer la gestión de estados de la orden y que la misma se resuelva en un momento posterior, lo puede hacer sin problemas.
2. No pueden comprarse acciones que no estén a la venta. Deben respetarse las cantidades y los montos límite. Si el usuario A quiere comprar NVDA a AR\$ 30.000 y el usuario B tiene para vender pero acepta mínimo AR\$ 35.000, no se puede hacer la transacción entre esas dos órdenes. Desde luego, la recomendación es que cuando creen la base de datos, generen portfolios que ya tengan acciones para poder realizar compras y ventas.
3. Si se ingresa una orden de compra por 40 acciones y hay una orden de venta por 50 acciones. La operación se puede realizar y debe actualizarse la orden de venta para que refleje que todavía puede vender 10 acciones.

3. Requerimientos Funcionales

El backend debe permitir, mínimamente:

1. Consultar cotizaciones (cotizaciones de acciones por su código o símbolo)
2. Consultar el portfolio y saldo de un usuario
3. Registrar y resolver órdenes de compra
4. Registrar y resolver órdenes de venta
5. Consultar el historial completo de un usuario (Todas las operaciones que realizó, ordenadas por fecha)
6. Consultar el historial completo de transacciones realizadas (para el usuario ADMIN)

4. Requerimientos de seguridad

1. Los usuarios deben ser Autenticados y Autorizados mediante el uso de OAuth2 y usando Keycloak como IDP
2. Debe existir un usuario ADMIN que sea capaz de consultar el historial de todas las operaciones realizadas.
3. El endpoint de consulta de cotizaciones es de acceso público

5. Requerimientos de Arquitectura

El sistema deberá desplegarse utilizando **Docker compose** y ser desarrollado mediante **Micro servicios**. Es su tarea decidir cuántos y cuáles serán los microservicios, siempre y cuando exista un único punto de entrada al backend. También es su tarea decidir si utilizan una base de datos por cada microservicio o una base de datos para todo el backend.

6. Calificación

- El TPI se evalúa con la exposición del grupo frente al docente y con el sistema funcionando.
- Los alumnos deben tener preparada una colección Postman (o cualquier herramienta similar) de manera de poder mostrar el funcionamiento durante la exposición.
- Los alumnos enfrentarán un coloquio por parte del docente donde se les preguntará sobre distintos aspectos de su aplicación.
- Se reserva un 15% de la nota para la implementación de algún apartado no cubierto en el curso de la materia (por ejemplo: caché, circuit breaker, métricas, logging, etc.)